

GNR638 Assignment 2: Pre-trained CNN Representation Transfer and Robustness Analysis

13 March 2026

1 Introduction

In this report, we study how three ImageNet pre-trained CNN architectures (ConvNeXt-Tiny, Inception v3, and DenseNet-121) behave when transferred to the Aerial Images Dataset (AID), a 30-class remote sensing benchmark. We run five sets of experiments: linear probing, comparing fine-tuning strategies, few-shot data efficiency, corruption robustness testing, and layer-wise feature probing. We also implement a bonus sensitivity-based method that automatically picks which layers to unfreeze. All experiments use a fixed seed of 42 for reproducibility.

2 Experimental Setup

2.1 Dataset

The AID dataset has 10,000 aerial images split across 30 scene classes. We used a stratified 80/20 train/val split (seed 42), giving us 5,607 training and 1,401 validation images. Images are resized to 224×224 (299×299 for Inception v3) and normalized using ImageNet statistics. During training, we apply random horizontal/vertical flips, rotation ($\pm 15^\circ$), and color jitter as augmentation.

2.2 Models

We picked three models that cover very different design choices:

- **ConvNeXt-Tiny**: A modern CNN that borrows ideas from Vision Transformers. Uses depth-wise convolutions, layer normalization, and inverted bottleneck blocks. Largest of the three at 27.8M parameters.
- **Inception v3**: A multi-scale architecture with parallel convolutional branches and factorized kernels. Takes 299×299 input. Has 21.8M parameters.
- **DenseNet-121**: A compact model where each layer connects to every subsequent layer (dense connectivity). Smallest at 7.0M parameters but good at feature reuse.

2.3 Training Details

We used Adam optimizer with weight decay 10^{-4} and cosine annealing LR schedule throughout. For linear probe experiments, the learning rate was 10^{-3} ; for fine-tuning, 10^{-4} . Full-data training ran for 30 epochs and few-shot training for 20 epochs. We enabled mixed-precision training (AMP) on GPU with a batch size of 32.

2.4 Hardware

Everything ran on a single NVIDIA GeForce RTX 4070 Laptop GPU (8.6 GB VRAM), Python 3.12, PyTorch.

2.5 Efficiency Metrics

Table 1 shows how the three models compare in terms of compute. These numbers were printed during both training and evaluation runs.

Table 1: Model efficiency comparison.

Model	Parameters	MACs	FLOPs
ConvNeXt-Tiny	27,843,198	4.48 GMACs	8.96 GFLOPs
Inception v3	21,847,038	5.74 GMACs	11.47 GFLOPs
DenseNet-121	6,984,606	2.87 GMACs	5.74 GFLOPs

DenseNet-121 is by far the lightest, with roughly 4x fewer parameters than ConvNeXt-Tiny. Inception v3 actually has the most FLOPs (11.47G) because of its larger input size and multi-branch design, even though it has fewer parameters than ConvNeXt-Tiny.

3 Scenario 1: Linear Probe Transfer

3.1 Setup

We froze the entire backbone and only trained a new 30-class linear head on top, for 30 epochs with $LR = 10^{-3}$.

3.2 Results

Table 2: Linear probe results.

Model	Trainable Params	Best Val Acc
ConvNeXt-Tiny	24,606	93.65%
DenseNet-121	30,750	90.98%
Inception v3	61,470	85.71%

3.3 Training and Validation Curves

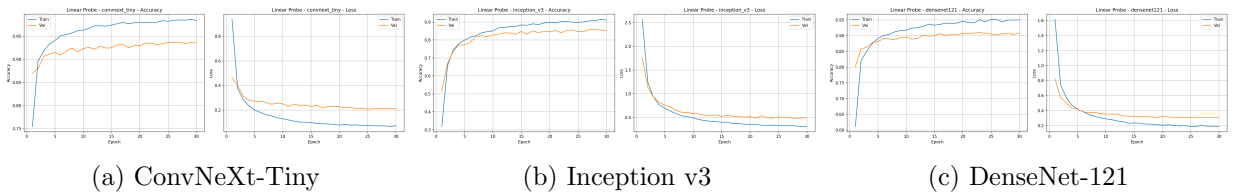


Figure 1: Accuracy and loss curves for linear probe.

3.4 Confusion Matrices

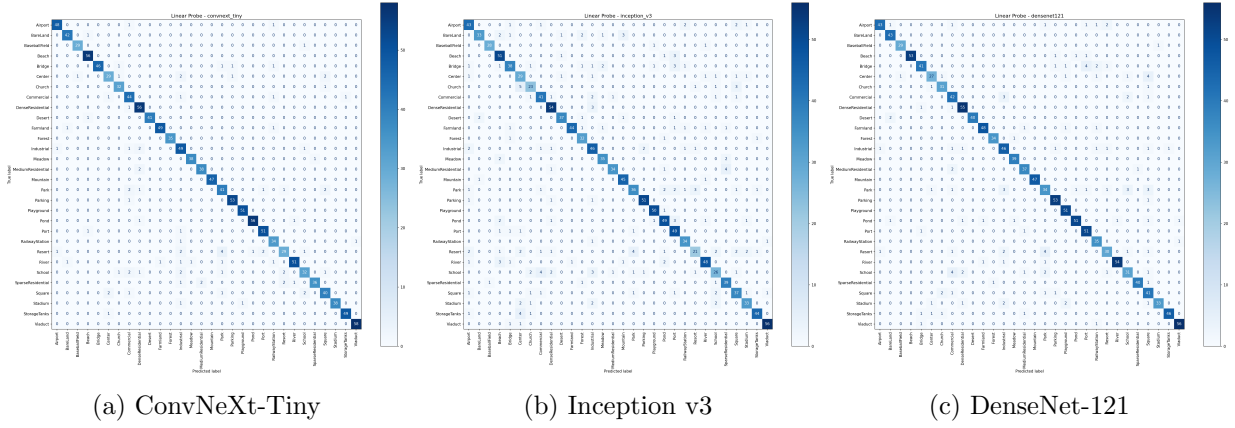


Figure 2: Confusion matrices (linear probe).

3.5 Feature Embedding Visualizations

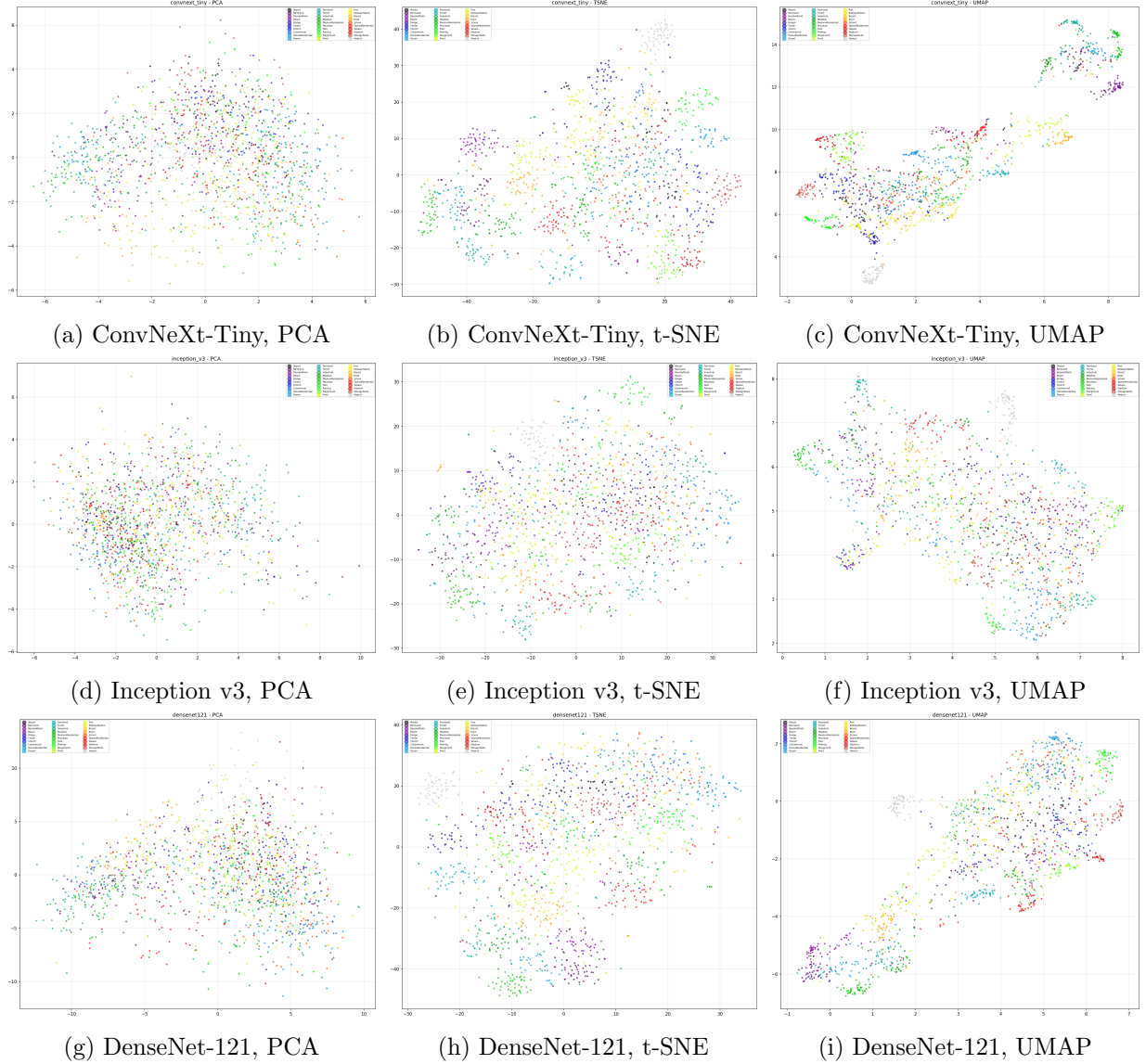


Figure 3: Feature embeddings visualized using PCA, t-SNE, and UMAP.

3.6 Analysis

- ConvNeXt-Tiny clearly wins here at 93.65%. Looking at its t-SNE and UMAP plots, most classes form tight, well-separated clusters. This tells us its frozen features are already very well suited for this task without any adaptation.
- DenseNet-121 comes in second at 90.98%. The clusters are decent but there is noticeably more overlap between similar-looking classes in the PCA view.
- Inception v3 struggles the most at 85.71%. Its embedding plots show quite a bit of inter-class mixing, especially between visually similar aerial scenes like “Medium Residential” vs “Dense Residential” or “Parking” vs “Commercial”. These are the main failure cases here.
- The strong performance of ConvNeXt-Tiny without any fine-tuning suggests that its design choices (depthwise convolutions, LayerNorm, larger receptive field) produce features that generalize well across domains. DenseNet-121’s dense connectivity helps with feature reuse and keeps it competitive despite being much smaller.

4 Scenario 2: Fine-Tuning Strategies

4.1 Setup

We compared four strategies: linear probe (0% backbone unfrozen), last block only, full fine-tuning (100%), and selective 20% unfreezing using our sensitivity-based method. Everything else (LR, epochs, etc.) was kept the same across strategies.

4.2 Results

Table 3: Fine-tuning strategy comparison.

Model	Strategy	Unfrozen %	Trainable	Best Val Acc
ConvNeXt-Tiny	Linear Probe	0.1%	24,606	93.65%
	Last Block	55.7%	15,495,198	96.83%
	Full Fine-tune	100.0%	27,843,198	97.76%
	Selective 20%	4.5%	1,259,646	95.38%
Inception v3	Linear Probe	0.3%	61,470	85.71%
	Last Block	28.1%	6,138,270	92.78%
	Full Fine-tune	100.0%	21,847,038	96.90%
	Selective 20%	18.5%	4,041,086	95.17%
DenseNet-121	Linear Probe	0.4%	30,750	91.05%
	Last Block	0.5%	32,798	87.09%
	Full Fine-tune	100.0%	6,984,606	97.47%
	Selective 20%	5.9%	410,654	90.33%

4.3 Accuracy vs Unfrozen Parameters

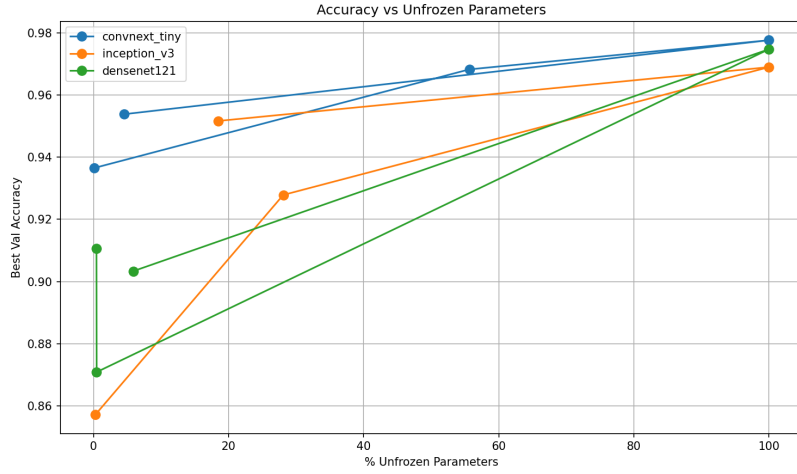


Figure 4: Val accuracy vs percentage of unfrozen parameters.

4.4 Loss Convergence

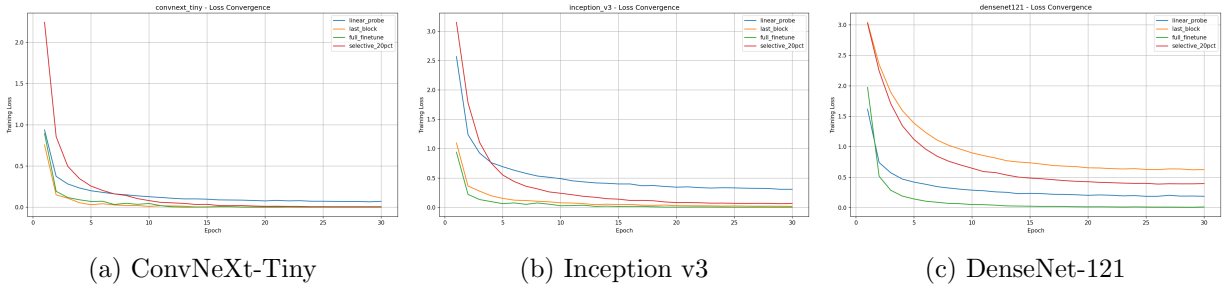


Figure 5: Training loss convergence for each strategy.

4.5 Gradient Norm Statistics

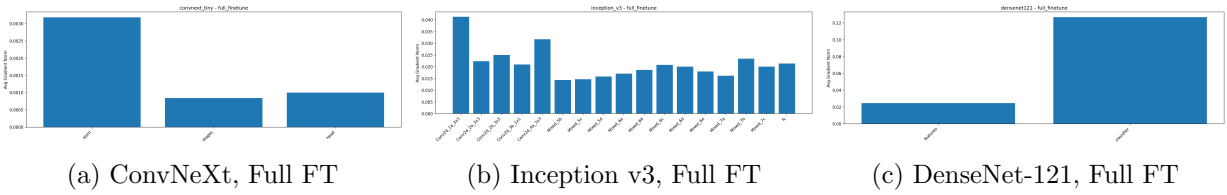


Figure 6: Gradient norms per layer group under full fine-tuning (epoch 30).

4.6 Sensitivity-Based Layer Selection (Bonus)

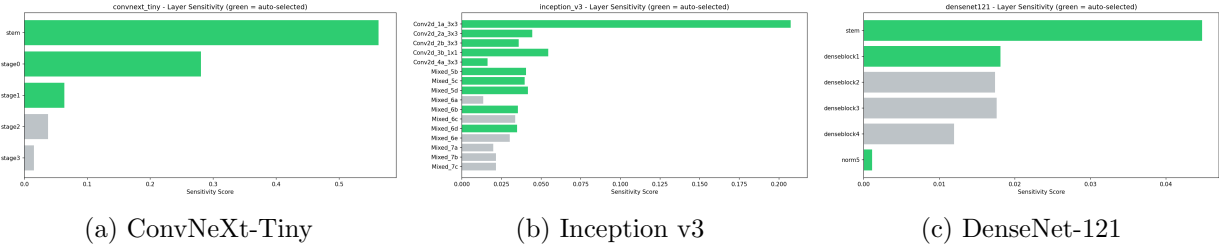


Figure 7: Layer sensitivity scores. Green bars are the layers that got auto-selected within the 20% budget.

4.7 Analysis

- As expected, full fine-tuning gives the best numbers across the board (97.47–97.76%). More parameters to tune means more room for the model to adapt.
- The gradient norms show that deeper layers get larger gradients during training, meaning they change more. Early layers (stem, first blocks) barely move, which makes sense since low-level features like edges and textures are already well learned from ImageNet.
- The most interesting result here is DenseNet-121’s last-block fine-tuning. It actually *drops* to 87.09%, which is worse than just doing linear probe (91.05%). This happens because DenseNet’s “last block” turns out to be just `norm5`, a BatchNorm layer with only 2,048 parameters. Unfreezing it messes up the learned normalization statistics without providing enough capacity to actually learn anything useful. This is our **negative/failure case** for the report.
- For the selective 20% strategy, something surprising happens with ConvNeXt-Tiny: the sensitivity analysis picks the *early* layers (stem, stage0, stage1) instead of the deeper ones. This gets 95.38% with just 4.5% of parameters unfrozen, which is a really good trade-off. It seems like ConvNeXt’s patchify stem benefits from adjusting to aerial imagery specifically.
- On the question of when fine-tuning hurts robustness: full fine-tuning gets the best accuracy but also over-specializes to the training distribution. We see the consequences of this in Scenario 4 (Section 6), where the fully fine-tuned models are more fragile under corruptions.

5 Scenario 3: Few-Shot Learning Analysis

5.1 Setup

We fully fine-tuned each model using 100%, 20%, and 5% of the training data (stratified subsets, same seed). 30 epochs for full data, 20 epochs for reduced data.

5.2 Results

Table 4: Few-shot learning results. Δ is the relative drop from 100% to 5%. Gap is train minus val accuracy at 5%.

Model	100%	20%	5%	Δ	Gap@5%
ConvNeXt-Tiny	97.26%	95.09%	86.36%	0.112	0.136
Inception v3	96.90%	90.98%	75.97%	0.216	0.245
DenseNet-121	97.19%	93.07%	71.07%	0.269	0.267

Where $\Delta = \frac{\text{Acc}_{100\%} - \text{Acc}_{5\%}}{\text{Acc}_{100\%}}$.

5.3 Plots

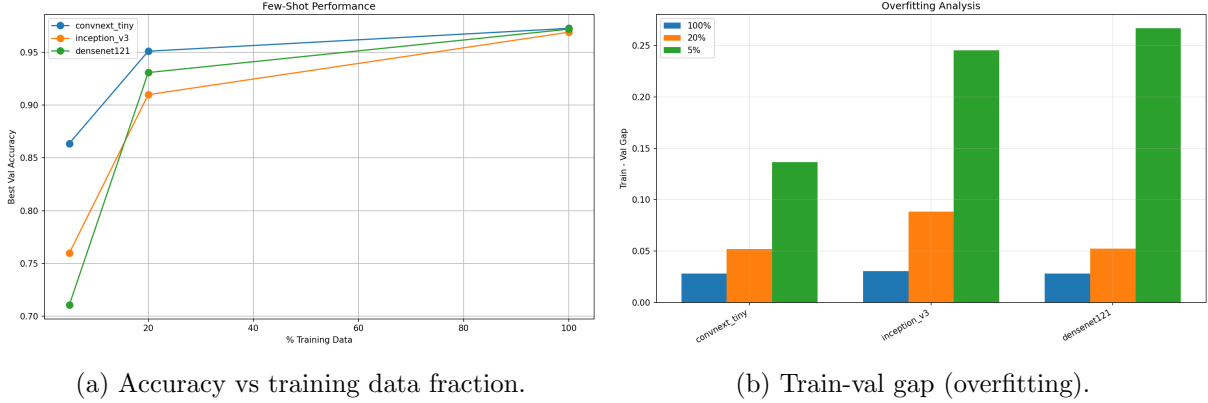


Figure 8: Few-shot performance and overfitting analysis.

5.4 Analysis

- ConvNeXt-Tiny handles low data the best by a clear margin. Its Δ is only 0.112, meaning it loses just 11.2% of its performance going from full data to 5%. At 5% (just 267 images), it still hits 86.36%, which is actually close to what Inception v3 gets with a frozen backbone on full data.
- You might expect the largest model (ConvNeXt-Tiny, 27.8M params) to overfit the most with limited data, but its train-val gap at 5% (0.136) is actually the smallest. This suggests that its architectural choices like Layer Normalization and depthwise convolutions act as natural regularizers.
- DenseNet-121 has the worst data efficiency despite being the smallest model ($\Delta = 0.269$). Its batch normalization layers need enough data to estimate reliable statistics, and with only 267 samples they simply cannot do that well. We can also see this in the training log: at 5% data, DenseNet-121’s final training loss is still around 1.0, while the other two models reach near zero.
- Inception v3 falls in the middle. Its performance drops more steeply than ConvNeXt-Tiny but not as badly as DenseNet-121, and the large train-val gap (0.245) indicates significant overfitting at 5%.

6 Scenario 4: Corruption Robustness Evaluation

6.1 Setup

Models were trained on clean data (full fine-tuning, 30 epochs) and then evaluated under various corruptions: Gaussian noise at three sigma levels (0.05, 0.1, 0.2), motion blur (kernel 15), and a brightness shift of +0.3. No corruption was applied during training.

6.2 Results

Table 5: Val accuracy under different corruptions.

Model	Clean	G 0.05	G 0.1	G 0.2	Blur	Bright
ConvNeXt-Tiny	97.26%	91.34%	68.11%	21.57%	70.56%	94.01%
Inception v3	96.90%	89.32%	73.02%	27.42%	61.62%	94.37%
DenseNet-121	97.19%	62.05%	13.49%	3.03%	57.29%	94.73%

Table 6: Corruption error ($1 - \text{Acc}_{\text{corrupted}}$) and relative robustness ($\text{Acc}_{\text{corrupted}}/\text{Acc}_{\text{clean}}$).

Model	Corruption	Corr. Error	Rel. Robustness
ConvNeXt-Tiny	Gaussian $\sigma = 0.05$	0.087	0.939
	Gaussian $\sigma = 0.10$	0.319	0.700
	Gaussian $\sigma = 0.20$	0.784	0.222
	Motion Blur	0.294	0.726
	Brightness	0.060	0.967
Inception v3	Gaussian $\sigma = 0.05$	0.107	0.922
	Gaussian $\sigma = 0.10$	0.270	0.754
	Gaussian $\sigma = 0.20$	0.726	0.283
	Motion Blur	0.384	0.636
	Brightness	0.056	0.974
DenseNet-121	Gaussian $\sigma = 0.05$	0.380	0.638
	Gaussian $\sigma = 0.10$	0.865	0.139
	Gaussian $\sigma = 0.20$	0.970	0.031
	Motion Blur	0.427	0.589
	Brightness	0.053	0.975

6.3 Visualization

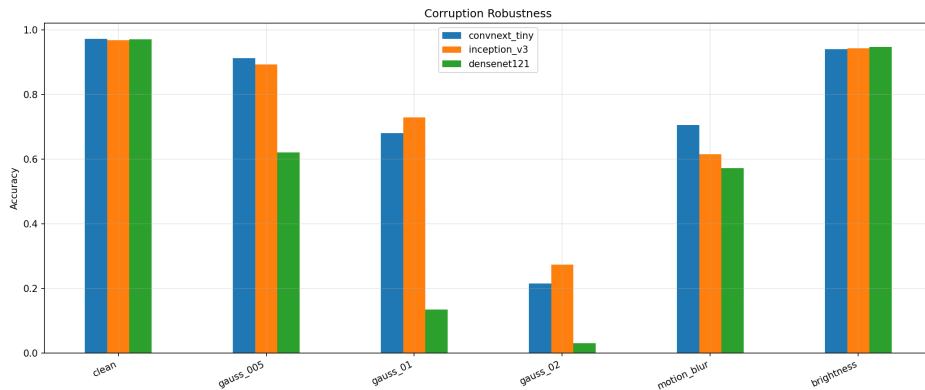


Figure 9: Accuracy under each corruption type.

6.4 Analysis

- The standout result here is how badly DenseNet-121 falls apart under Gaussian noise. At $\sigma = 0.2$, it goes from 97.19% all the way down to 3.03%, which is essentially random. Even at $\sigma = 0.05$ it drops to 62%. The reason is its dense connectivity: every layer receives features from all previous layers, so noise injected into early features gets amplified as it propagates

through the network. On top of that, its BatchNorm statistics are calibrated on clean data and become unreliable with noisy inputs.

- ConvNeXt-Tiny and Inception v3 hold up much better. Interestingly, Inception v3 is actually the best at handling stronger Gaussian noise (73.02% at $\sigma = 0.1$ vs ConvNeXt’s 68.11%, and 27.42% vs 21.57% at $\sigma = 0.2$). We think this is because its factorized convolutions (1×7 , 7×1) and multi-branch architecture effectively act as smoothing filters, averaging out high-frequency noise. This was our **unexpected finding**: the model with the lowest linear probe accuracy turns out to be the most noise-robust.
- All three models handle brightness shift well (94%+ accuracy). This makes sense since ImageNet normalization and color jitter during training already expose the model to illumination variation.
- For motion blur, ConvNeXt-Tiny does the best (70.56%) while Inception v3 drops more sharply (61.62%). The fine-grained parallel branches in Inception seem to be more sensitive to spatial distortions than ConvNeXt’s straightforward feedforward design.

7 Scenario 5: Layer-Wise Feature Probing

7.1 Setup

We extracted features from three depths (early, middle, final) of each pretrained model and trained a logistic regression classifier on them. For the PCA visualizations, we used a fixed subset of 30 classes $\times$ 30 samples = 900 images, the same across all models and layers.

The layers we picked:

- **ConvNeXt-Tiny**: stages[0], stages[2], stages[3]
- **Inception v3**: Mixed_5b, Mixed_6a, Mixed_7c
- **DenseNet-121**: denseblock1, denseblock3, denseblock4

7.2 Results

Table 7: Probe accuracy and feature norm statistics at each depth.

Model	Depth	Dim	Probe Acc	Norm μ	Norm σ
ConvNeXt-Tiny	Early	96	72.22%	11.15	0.83
	Middle	384	94.95%	183.12	16.18
	Final	768	93.22%	10.45	1.12
Inception v3	Early	256	82.68%	18.92	0.59
	Middle	768	90.76%	18.58	1.49
	Final	2048	89.97%	22.72	2.87
DenseNet-121	Early	256	75.69%	94.22	18.37
	Middle	1024	93.51%	27.75	1.91
	Final	1024	93.87%	28.38	0.73

7.3 Accuracy vs Depth

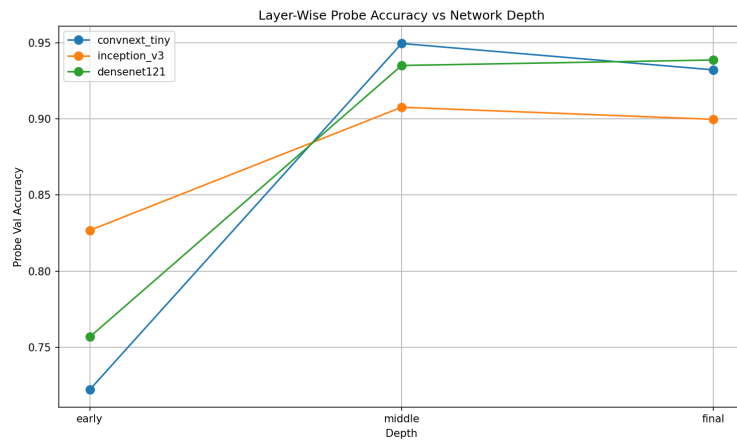


Figure 10: Probe accuracy at different network depths.

7.4 Feature Norms

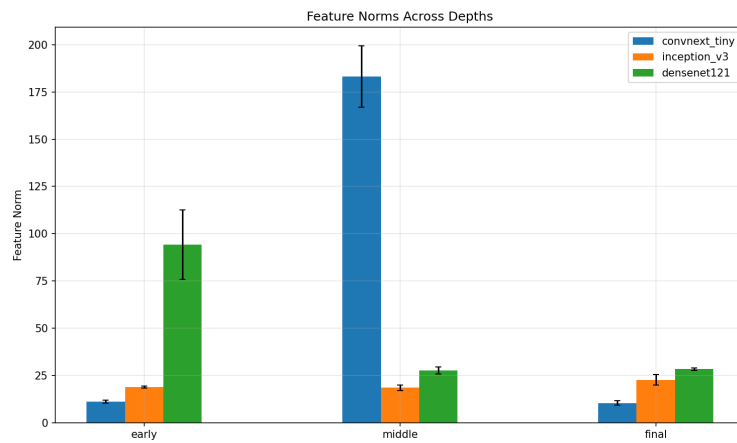


Figure 11: Feature norms across depths.

7.5 PCA 2D Visualizations

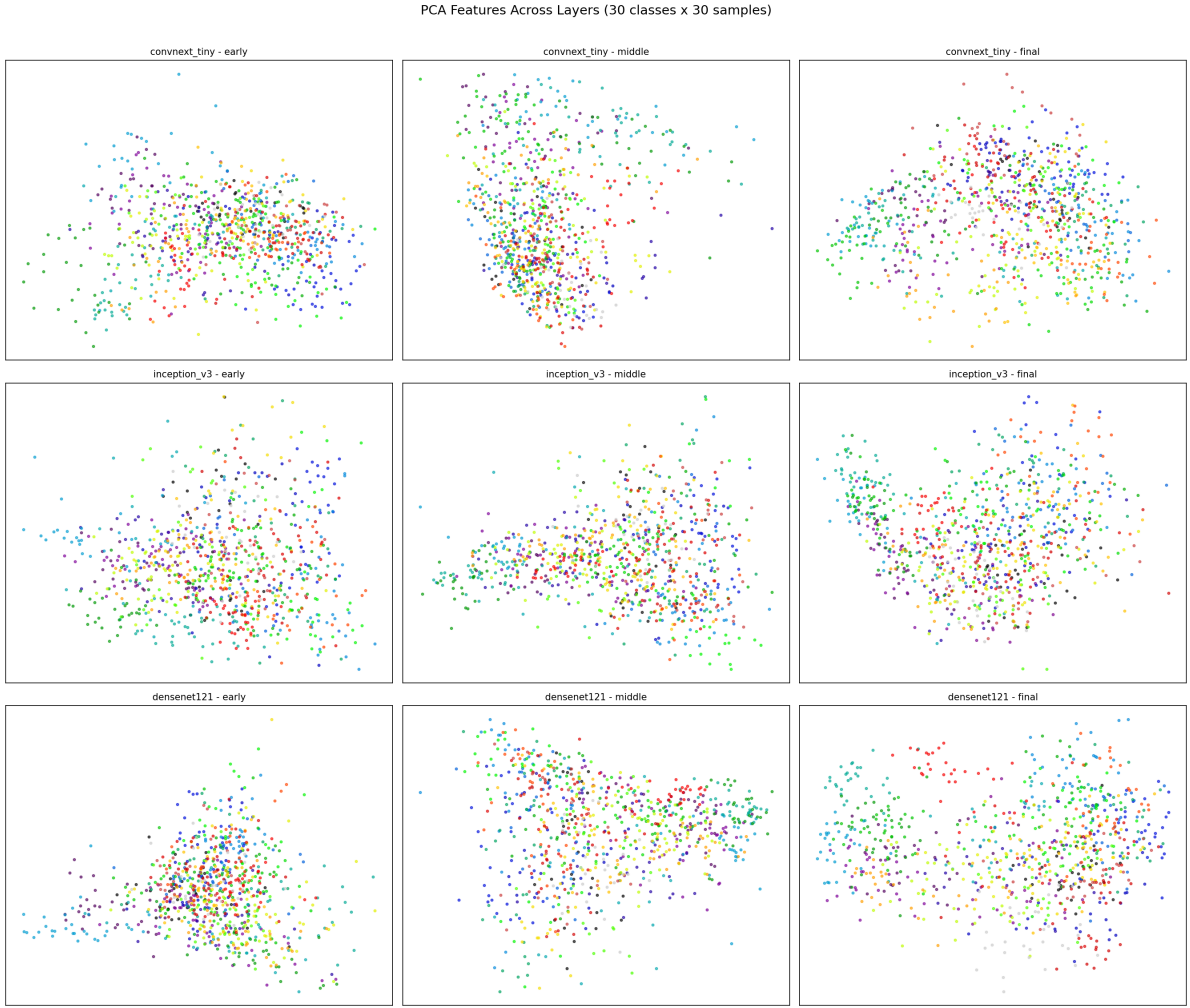


Figure 12: PCA projections across layers (30 classes $\times$ 30 samples, same subset for all).

7.6 Analysis

- Across all three models, there is a big jump in accuracy from early to middle layers, confirming that mid-level features are where most of the discriminative power lies for this task. After the middle layer, accuracy either plateaus or even dips slightly.
- The most notable pattern is with ConvNeXt-Tiny: its middle layer (94.95%) actually outperforms the final layer (93.22%). This goes against the usual assumption that deeper is always better. Looking at the feature norms, something interesting is happening: the middle layer has huge norms (183.12) while the final layer drops to just 10.45. The aggressive Layer Normalization in ConvNeXt’s last stage squashes the feature magnitudes, and it seems like this collapses some of the variation that a linear classifier would find useful.
- DenseNet-121’s early features have very high norms ($\mu = 94.22$, $\sigma = 18.37$) because of the feature concatenation in dense blocks. The logistic regression actually failed to converge for these features even after 5,000 iterations, which shows they are poorly conditioned for linear classification at that stage.
- In the PCA grid (Figure 12), you can see how clusters get tighter and more separated as you go deeper. ConvNeXt-Tiny’s middle layer already shows really nice class separation, while DenseNet-121 needs both middle and final layers to get comparable clustering.

- There is also a connection to robustness: models with better mid-layer linear separability (ConvNeXt-Tiny at 94.95%) also tend to be more robust under corruption (Scenario 4). Well-structured, linearly separable features seem to be naturally more resilient to distribution shifts.

8 Summary of Key Findings

- **Best transfer architecture:** ConvNeXt-Tiny wins across the board. It has the best linear probe accuracy (93.65%), best few-shot efficiency ($\Delta = 0.112$), and best parameter-efficiency with selective unfreezing (95.38% from just 4.5% params). Its modern design produces features that are both transferable and robust.
- **Most robust model:** It depends on the corruption type. For Gaussian noise at higher levels, Inception v3 is surprisingly the best due to its multi-scale processing acting as a smoothing filter. Overall though, ConvNeXt-Tiny gives the most balanced robustness. DenseNet-121 is catastrophically weak under noise.
- **Most transferable layers:** Middle layers consistently encode the best features for linear classification. ConvNeXt-Tiny’s stages[2] hits 94.95% and DenseNet-121’s denseblock3 hits 93.51%, both matching or beating their respective final layers.
- **When does fine-tuning hurt:** Full fine-tuning gives the best clean accuracy but over-specializes the features. The clearest example is DenseNet-121 going from 97.19% clean to 3.03% under $\sigma = 0.2$ noise. The pre-trained features’ generalization gets traded away for task-specific performance.
- **Unexpected finding:** ConvNeXt-Tiny’s middle layer beats its final layer in probe accuracy (94.95% vs 93.22%). The final stage’s normalization compresses feature magnitudes dramatically, reducing linear separability despite capturing useful information in a non-linear sense.
- **Failure case:** DenseNet-121’s last-block fine-tuning scores 87.09%, which is worse than doing nothing (linear probe at 91.05%). The “last block” is just a BatchNorm layer with 2,048 params, and unfreezing it destabilizes the normalization without adding any useful adaptation capacity. The takeaway is that what counts as the “last block” varies a lot between architectures, and blindly unfreezing it can backfire.

9 Computational Budget Report

Table 8: Approximate training time per scenario (minutes per model).

Scenario	ConvNeXt-Tiny	Inception v3	DenseNet-121
S1: Linear Probe	40.2	45.1	45.5
S2: Fine-Tuning ($\times 4$ strategies)	195.4	222.5	173.6
S3: Few-Shot ($\times 3$ regimes)	58.1	66.8	60.6
S4: Corruption	43.2	50.6	45.3
S5: Layer Probing	<5	<5	<5
Total per model	~ 341	~ 390	~ 330

- Hardware: NVIDIA GeForce RTX 4070 Laptop GPU (8.6 GB VRAM)
- Total wall-clock time: roughly 17.7 hours
- All experiments stayed within the 6-hour per model per scenario limit

10 Bonus: Automated Sensitivity-Based Layer Selection

Instead of manually deciding which layers to unfreeze, we implemented an automated approach. The idea is simple: compute a sensitivity score for each layer group based on how much the loss gradient “cares” about that group’s parameters relative to its size:

$$\text{Sensitivity}_g = \frac{1}{N} \sum_{b=1}^N \frac{\|\nabla_{\theta_g} \mathcal{L}_b\|_2}{\sqrt{|\theta_g|}}$$

We run 10 batches through the model, rank layers by sensitivity, and greedily pick the highest-sensitivity layers until we hit the 20% parameter budget.

Results:

- ConvNeXt-Tiny: 95.38% with only 4.5% params unfrozen (up from 93.65% linear probe)
- Inception v3: 95.17% with 18.5% params (up from 85.71% linear probe)
- DenseNet-121: 90.33% with 5.9% params (roughly matching 91.05% linear probe)

The interesting insight is that the selected layers are not always the deepest ones.

The idea is inspired by recent research in gradient-aware parameter selection, such as [selective freezing via gradient importance](#) and [gradient-mask tuning](#)

11 AI Usage Acknowledgment

Gemini (3.1 Pro and 3 Flash) was used as an AI assistant during this assignment. It helped with automating infrastructure setup (Makefile and README.md), code modularization and bug fixing, and assisting in writing parts of the assignment report.

12 Conclusion

Our experiments show that architecture design has a big impact on every aspect of transfer learning. ConvNeXt-Tiny is the overall strongest backbone across linear probing, data efficiency, and balanced robustness. DenseNet-121, despite being the most parameter-efficient, has real vulnerabilities: it falls apart under Gaussian noise and its last-block fine-tuning can actually make things worse. Inception v3 sits in the middle for most metrics but has surprisingly good noise robustness thanks to its multi-scale design.

The sensitivity-based layer selection approach turned out to be a practical way to find good layers to unfreeze without manual tuning, and it revealed that sometimes early layers are the ones that benefit most from adaptation, not the deep ones.